Custom Types Validator

Using a typed redeemer with custom data in Plutus

Aman Kumar
1/1/2025 • 6 min read

plutus cardano validator typed custom-types

Share

← Back to Home

INTRODUCTION

Custom Types Validator demonstrates how to use*custom, typed redeemers* in Plutus instead of raw integers or `BuiltinData`
. This approach improves correctness, safety, and clarity by encoding business rules directly into Haskell types. In this example, funds locked at the script address can only be spent when the redeemer is exactly **MkMySillyRedeemer 42**.

CustomTypes.hs

Copy Export

Markdown (.md)
PDF (.pdf)

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
```

Related

O One-Shot Minting Policy
Mint exactly once using a ...

E Exploitable Swap
A naive ADA-for-token sw...

S Staking Validator
Reward distribution enforc...

plutus-cheatsheet-gener

plutus-cheatsheet.vercel.app/article/custom-types-validator

Incognito

All Bookmarks

Custom Types Validator

Using a typed redeemer with custom data in Plutus

Aman Kumar
1/1/2025 • 6 min read

plutus cardano validator typed custom-types

Share

Cancel Name CustomTypes.hs.md Save

Home
Docume...
Downloa...
Music
Pictures
Videos
Other Lo...

raj Music

Name	Size	Type	Modified
Test			26 Dec 2025
my-project			12 Dec 2025

*.md

```haskell
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
```

plutus-cheatsheet-gener
plutus-cheatsheet.vercel.app/article/custom-types-validator
Incognito
All Bookmarks
Anyone using this device can see downloaded files
CustomTypes.hs.md
2,009 B • Done

Custom Types Validator

Using a typed redeemer with custom data in Plutus

Aman Kumar
1/1/2025 • 6 min read

plutus cardano validator typed custom-types

Share

← Back to Home

INTRODUCTION

Custom Types Validator demonstrates how to use*custom, typed redeemers* in Plutus instead of raw integers or `BuiltinData`
. This approach improves correctness, safety, and clarity by encoding business rules directly into Haskell types. In this example, funds locked at the script address can only be spent when the redeemer is exactly **MkMySillyRedeemer 42**.

CustomTypes.hs

Copy Export

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
```

Related

O One-Shot Minting Policy
Mint exactly once using a ...

E Exploitable Swap
A naive ADA-for-token sw...

S Staking Validator
Reward distribution enforc...

Open

CustomTypes.hs.md
~/Music


```haskell
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
import           Prelude              (IO)
import           Utilities            (wrapValidator, writeValidatorToFile)

-------------------------------------------------------------------------------------------
---------------------------------- ON-CHAIN / VALIDATOR ----------------------------------------

-- We can create custom data types for our datum and redeemer like this:
newtype MySillyRedeemer = MkMySillyRedeemer Integer
PlutusTx.unstableMakeIsData ''MySillyRedeemer

-- This validator succeeds only if the redeemer is `MkMySillyRedeemer 42`
--              Datum     Redeemer            ScriptContext
mkCTValidator :: () -> MySillyRedeemer -> PlutusV2.ScriptContext -> Bool
mkCTValidator _ (MkMySillyRedeemer r) _ =
  traceIfFalse "expected 42" $ r == 42
{-# INLINABLE mkCTValidator #-}

wrappedMkVal :: BuiltinData -> BuiltinData -> BuiltinData -> ()
wrappedMkVal = wrapValidator mkCTValidator
{-# INLINABLE wrappedMkVal #-}

validator :: PlutusV2.Validator
```

Custom Types Validator

Using a typed redeemer with custom data in Plutus

Aman Kumar
1/1/2025 • 6 min read

plutus cardano validator typed custom-types

Share

← Back to Home

INTRODUCTION

Custom Types Validator demonstrates how to use*custom, typed redeemers* in Plutus instead of raw integers or `BuiltinData`
. This approach improves correctness, safety, and clarity by encoding business rules directly into Haskell types. In this example, funds locked at the script address can only be spent when the redeemer is exactly **MkMySillyRedeemer 42**.

CustomTypes.hs

Copy Export

Markdown (.md)
PDF (.pdf)

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
```

Related

O One-Shot Minting Policy
Mint exactly once using a ...

E Exploitable Swap
A naive ADA-for-token sw...

S Staking Validator
Reward distribution enforc...

plutus-cheatsheet-gener

plutus-cheatsheet.vercel.app/article/custom-types-validator

Incognito

All Bookmarks

Custom Types Validator

Using a typed redeemer with custom data in Plutus

Aman Kumar
1/1/2025 • 6 min read

plutus cardano validator typed custom-types

Share

Cancel Name CustomTypes.hs.pdf Save

Home
Docume...
Downloa...
Music
Pictures
Videos
Other Lo...

raj Music

Name	Size	Type	Modified
Test			26 Dec 2025
my-project			12 Dec 2025

*.pdf

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
```

plutus-cheatsheet-gener
plutus-cheatsheet.vercel.app/article/custom-types-validator
Incognito
All Bookmarks
Anyone using this device can see downloaded files
CustomTypes.hs.pdf
7.9 KB • Done

Custom Types Validator

Using a typed redeemer with custom data in Plutus

Aman Kumar
1/1/2025 • 6 min read

plutus cardano validator typed custom-types

Share

← Back to Home

INTRODUCTION

Custom Types Validator demonstrates how to use*custom, typed redeemers* in Plutus instead of raw integers or `BuiltinData`
. This approach improves correctness, safety, and clarity by encoding business rules directly into Haskell types. In this example, funds locked at the script address can only be spent when the redeemer is exactly **MkMySillyRedeemer 42**.

CustomTypes.hs

Copy Export

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
```

Related

O One-Shot Minting Policy
Mint exactly once using a ...

E Exploitable Swap
A naive ADA-for-token sw...

S Staking Validator
Reward distribution enforc...

plutus-cheatsheet-genera | CustomTypes.hs.pdf

File /home/raj/Music/CustomTypes.hs.pdf

Incognito

All Bookmarks

CustomTypes.hs.pdf 1 / 1 100%

1

```haskell
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
import           Prelude              (IO)
import           Utilities            (wrapValidator, writeValidatorToFile)

-------------------------------------------------------------------------------
--------------
---------------------------------- ON-CHAIN / VALIDATOR
----------------------------------------

-- We can create custom data types for our datum and redeemer like this:
newtype MySillyRedeemer = MkMySillyRedeemer Integer
PlutusTx.unstableMakeIsData ''MySillyRedeemer

-- This validator succeeds only if the redeemer is `MkMySillyRedeemer 42`
--              Datum     Redeemer            ScriptContext
mkCTValidator :: () -> MySillyRedeemer -> PlutusV2.ScriptContext -> Bool
mkCTValidator _ (MkMySillyRedeemer r) _ =
  traceIfFalse "expected 42" $ r == 42
{-# INLINABLE mkCTValidator #-}

wrappedMkVal :: BuiltinData -> BuiltinData -> BuiltinData -> ()
wrappedMkVal = wrapValidator mkCTValidator
{-# INLINABLE wrappedMkVal #-}

validator :: PlutusV2.Validator
validator =
  PlutusV2.mkValidatorScript
    $$(PlutusTx.compile [|| wrappedMkVal ||])
```